

Laporan Milestone 3 Tugas Besar

IF2224 Teori Bahasa Formal dan Automata

Arion Compiler



K01 | Kelompok MLB

13524009 Mikhael Benrael Tampubolon

13524011 Muhammad Iqbal Raihan

13524025 Moh Hafizh Irham Perdana

13524099 Muhammad Akmal

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

Daftar Isi

Daftar Isi.....	2
BAB 1.....	3
1.1. Abstract Syntax Tree.....	3
1.2. Symbol Table.....	4
1.3. Semantic Analysis.....	8
BAB 2.....	13
2.1. Gambaran Umum Progam.....	13
2.2. Teknologi yang Digunakan.....	13
2.3. Struktur Program.....	13
2.4. Fungsi dan Kelas Utama.....	14
2.5. Alur Kerja Progam.....	16
BAB 3.....	17
3.1. Implementasi Parser Utama.....	17
3.2. Implementasi Parse Tree.....	17
3.3. Implementasi Modul Header dan Deklarasi Program.....	17
3.4. Implementasi Modul Subprogram.....	17
BAB 4.....	19
4.1. Pengujian ke-1.....	19
4.2. Pengujian ke-2.....	19
4.3. Pengujian ke-3.....	19
4.4. Pengujian ke-3.....	19
4.5. Pengujian ke-5.....	20
BAB 5.....	21
Referensi.....	25

BAB 1

TEORI DASAR

1.1. Abstract Syntax Tree

Abstract Syntax Tree (AST) adalah struktur data pohon yang merepresentasikan struktur sintaks abstrak dari *source code* suatu program. AST merupakan bentuk ringkas dari *parse tree* yang dihasilkan oleh *parser*.

Perbedaan AST dengan *parse tree* terletak pada abstraksinya. AST membuang informasi-informasi yang tidak relevan untuk analisis semantik seperti tanda kurung, kata kunci terminal, dan node non-terminal. AST berfokus pada hubungan semantik antar elemen program. Sebagai contoh, untuk ekspresi $7 + (2 + 3)$, *parse tree* akan menghasilkan banyak node non-terminal seperti *expr*, *term*, dan *factor* yang hanya berperan sebagai perantara derivasi. Sementara itu, AST dari ekspresi yang sama cukup direpresentasikan dengan tiga node: satu node operator $+$ dengan dua child, yaitu node konstanta 7 dan node operator $+$ lainnya yang memiliki child 2 dan 3.

Pembentukan AST dari *parse tree* dilakukan melalui mekanisme Syntax-Directed Translation (SDT). SDT adalah metode yang menyisipkan semantic action ke dalam aturan produksi grammar, sehingga setelah *parse tree* terbentuk, builder menelusuri node *parse tree* dan menerapkan *semantic action*.

Dalam SDT, setiap simbol grammar dapat memiliki atribut. Terdapat dua jenis atribut berdasarkan arah informasinya. Synthesized attribute adalah atribut yang nilainya dihitung dari atribut child-nya, sehingga informasi mengalir dari bawah ke atas (bottom-up). Inherited attribute adalah atribut yang nilainya diturunkan dari parent atau sibling kirinya, sehingga informasi mengalir dari atas ke bawah atau dari kiri ke kanan.

Pada Milestone 3, algoritma yang digunakan adalah L-Attributed Grammar, yaitu subset dari Attributed Grammar yang memperbolehkan penggunaan baik synthesized attribute maupun inherited attribute, dengan syarat bahwa inherited attribute suatu simbol hanya boleh bergantung pada atribut dari parent atau sibling di sebelah kirinya, tidak dari sibling di sebelah kanannya. Properti ini memungkinkan evaluasi atribut dilakukan dalam satu kali traversal dari kiri ke kanan (*left-to-right*), sehingga cocok dengan strategi top-down parsing yang digunakan.

Proses pembentukan AST dilakukan dengan cara: (1) membentuk node baru untuk setiap konstruksi semantik yang ditemukan; (2) menghilangkan informasi sintaksis yang tidak diperlukan; serta (3) menggabungkan potongan-potongan node dari level bawah ke atas hingga terbentuk satu AST yang utuh yang kemudian akan diubah menjadi Decorated AST.

Decorated AST adalah AST yang telah dianotasi dengan informasi semantik sebagai hasil dari proses Semantic Analysis. Anotasi yang ditambahkan pada setiap node mencakup tiga hal utama: tipe data akhir dari ekspresi yang direpresentasikan oleh node tersebut, referensi atau indeks yang mengarah pada entri yang tepat di dalam *symbol table*, serta *lexical level* atau tingkatan *scope* tempat node tersebut berada.

1.2. Symbol Table

Symbol table adalah struktur data yang digunakan oleh compiler untuk menyimpan dan mengelola informasi mengenai setiap identifier yang dideklarasikan dalam program. Informasi ini mencakup nama identifier, tipe datanya, kelas objeknya (variabel, konstanta, prosedur, fungsi, atau tipe), lexical level tempat ia dideklarasikan, dan informasi lain yang relevan untuk pengecekan semantik maupun generasi kode.

Peran utama *symbol table* adalah meneruskan informasi dari deklarasi ke penggunaan identifier. Ketika suatu deklarasi diproses, compiler memasukkan informasi identifier tersebut ke dalam *symbol table*, seperti nama, kelas objek, tipe data, lexical level, dan atribut lain yang diperlukan. Ketika identifier yang sama digunakan di bagian program lain, compiler melakukan lookup ke *symbol table* untuk mengambil kembali informasi tersebut. Dengan demikian, *symbol table* menjadi penghubung antara proses *declaration checking* dan penggunaan identifier dalam ekspresi maupun statement.

Pada implementasi compiler tugas besar ini, *symbol table* terdiri dari tiga tabel yang saling berkaitan, yaitu *tab*, *btab*, dan *atab*. Ketiga tabel ini dikelola seperti stack: saat memasuki blok baru (prosedur, fungsi, atau record), entri baru ditambahkan ke tabel yang relevan, dan saat keluar dari blok, informasi tersebut tidak lagi dapat diakses dari *scope* yang lebih luar.

a. Tabel *tab*

Tabel `tab` adalah tabel utama yang menyimpan informasi setiap identifier dalam program, termasuk konstanta, variabel, tipe, prosedur, dan fungsi. Indeks pada tabel ini dimulai dari 33, karena indeks 0 hingga 32 digunakan untuk *reserved words* dan *predefined identifiers* yang sudah diinisialisasi sebelum proses analisis dimulai.

Atribut	Deskripsi
<code>identifiers</code>	Nama identifier. Indeks dimulai dari 33 karena indeks 0–32 digunakan untuk reserved words dan predefined identifiers.
<code>link</code>	Pointer/indeks ke identifier sebelumnya dalam scope yang sama. Membentuk linked list per blok untuk manajemen scope.
<code>obj</code>	Kelas objek yang dienumerasi: konstanta, variabel, tipe, prosedur, fungsi, dan sebagainya.
<code>type</code>	Tipe dasar identifier, misalnya integer, boolean, char, real, array, record. Berupa kode numerik.
<code>ref</code>	Pointer ke tabel lain jika tipe adalah komposit. Mengarah ke atab (array) atau btab (record/prosedur).
<code>nrm</code>	Menandai apakah identifier adalah variabel normal (=1) atau parameter by-reference/var parameter (=0).
<code>lev</code>	Lexical level tempat identifier dideklarasikan. 0 = global, 1 = dalam prosedur/fungsi, 2 = bersarang lebih dalam, dst.
<code>adr</code>	Alamat atau offset. Maknanya tergantung jenis objek: offset variabel di stack frame, nilai konstanta, atau alamat prosedur.

Atribut `link` pada `tab` berperan penting dalam manajemen *scope*. Setiap identifier memiliki `link` yang menunjuk ke identifier sebelumnya dalam blok yang sama, membentuk struktur *linked list* per blok. Saat melakukan lookup, compiler menelusuri linked list ini dari identifier terakhir yang dideklarasikan mundur ke awal blok, kemudian melanjutkan ke blok yang lebih luar jika tidak ditemukan.

b. Tabel `btab`

Tabel `btab` menyimpan informasi mengenai setiap blok dalam program, yaitu prosedur, fungsi, maupun definisi tipe record. Setiap blok memiliki entri

tersendiri di `btab` yang mencatat batas identifier di dalamnya serta ukuran memori yang diperlukan.

Atribut	Deskripsi
<code>block</code>	Indeks entri block (setiap block mewakili prosedur, fungsi, atau record type definition).
<code>last</code>	Pointer ke identifier terakhir yang dideklarasikan di dalam blok (field record, parameter, atau variabel lokal).
<code>lpar</code>	Pointer ke parameter terakhir dari prosedur/fungsi pada blok ini, Jika blok adalah record, nilainya 0
<code>psze</code>	Total ukuran parameter blok dalam byte/unit memori
<code>vsze</code>	Total ukuran variabel lokal blok dalam byte/unit memori

Ketika compiler memasuki blok baru, entri baru ditambahkan ke `btab` dan `display[level]` diperbarui untuk menunjuk ke entri tersebut. Atribut `last` pada `btab` menjadi titik awal untuk menelusuri semua identifier yang dideklarasikan dalam blok tersebut mengikuti rantai `link` di `tab`.

c. Tabel atab

Tabel `atab` menyimpan informasi detail mengenai setiap tipe array yang dideklarasikan dalam program. Saat sebuah identifier bertipe array ditemukan, tabel `tab` akan mencatat tipe sebagai array dan atribut `ref` akan menunjuk ke entri yang bersesuaian di `atab`.

Atribut	Deskripsi
<code>arrays</code>	Indeks entri array
<code>xtyp</code>	Tipe indeks array, misalnya integer. Berupa kode tipe dari tabel <code>tab</code> .
<code>etyp</code>	Tipe elemen array, misalnya integer atau record. Berupa kode tipe dari tabel <code>tab</code> .
<code>eref</code>	Pointer ke detail tipe elemen jika elemen adalah tipe komposit (array dalam array, atau record) yang mengarah ke <code>atab</code> atau <code>btab</code> .
<code>low</code>	Batas bawah indeks array, misalnya 1 pada <code>array[1..10]</code> atau 0 pada <code>array[0..9]</code>

high	Batas atas indeks array
elsz	Ukuran satu elemen array dalam byte/unit memori
size	Total ukuran array secara keseluruhan

Selain itu terdapat juga manajemen scope (ruang lingkup) yang menentukan di bagian mana suatu identifier dapat diakses dalam program. Bahasa Arion menggunakan *static scoping (lexical scoping)*, di mana scope suatu identifier ditentukan berdasarkan struktur teks program, bukan alur eksekusi. Setiap blok (prosedur, fungsi, atau blok utama program) menciptakan scope baru yang bersifat bersarang di dalam scope yang memuatnya.

Lexical level (lev) merefleksikan kedalaman bersarang suatu identifier. Level 0 adalah scope global tempat deklarasi program utama berada. Level 1 adalah scope di dalam prosedur atau fungsi yang dideklarasikan langsung di level global, dan seterusnya untuk setiap tingkat bersarang berikutnya.

Sebelum proses analisis semantik dimulai, symbol table diinisialisasi terlebih dahulu dengan sejumlah identifier bawaan (*predefined identifier*) yang sudah tersedia tanpa perlu dideklarasikan oleh pengguna. Predefined identifier ini mencakup tipe-tipe dasar bahasa Arion serta prosedur standar untuk input dan output.

Identifier	Keterangan
Integer	Digunakan untuk deklarasi variabel dan parameter bertipe bilangan bulat.
Real	Digunakan untuk bilangan desimal.
Char	Digunakan untuk karakter tunggal.
Boolean	Digunakan untuk tipe boolean.
String	Digunakan untuk teks.
True	Konstanta Boolean yang bernilai true
False	Konstanta Boolean yang bernilai false.
Writeln	Prosedur untuk mencetak output ke terminal dengan new line.
Readln	Prosedur untuk membaca input dari terminal.

1.3. Semantic Analysis

Semantic Analysis adalah tahap ketiga dalam pipeline compiler, yang berlangsung setelah *lexical analysis* dan *syntax analysis*. Pada tahap ini, compiler memeriksa kebenaran makna atau semantik program, bukan hanya kebenaran strukturnya. Semantic Analysis menerima parse tree sebagai masukan dan menghasilkan Decorated AST beserta symbol table yang terisi sebagai keluaran. Terdapat empat aspek utama yang diperiksa dalam Semantic Analysis, yaitu Type Checking, Symbol Table Management, Scope Resolution, dan Control Flow Validation.

Pemeriksaan yang dilakukan pada tahap ini termasuk ke dalam static checking, yaitu pengecekan yang dilakukan pada saat kompilasi tanpa menjalankan program. Static checking memungkinkan compiler mendeteksi kesalahan seperti *type mismatch*, penggunaan identifier yang belum dideklarasikan, pelanggaran aturan scope, dan penggunaan operator yang tidak sesuai dengan tipe operand. Dengan melakukan pengecekan ini sebelum program dijalankan, compiler dapat mencegah sejumlah kesalahan semantik sejak tahap kompilasi.

a. Type Checking dan Type Compatibility

Type checking memastikan bahwa setiap operasi dalam program dilakukan pada operand dengan tipe yang sesuai. Setiap node expression pada AST dihitung tipe datanya berdasarkan tipe operand-nya dan operator yang digunakan. Hasil penghitungan tipe ini kemudian digunakan untuk memverifikasi kompatibilitas dalam konteks penggunaannya, misalnya pada assignment atau pemanggilan prosedur.

Dalam pemeriksaan tipe ekspresi, Arion Compiler terutama menggunakan pendekatan *type synthesis*. Type synthesis berarti tipe suatu ekspresi dihitung dari tipe sub-ekspresi atau operand penyusunnya. Sebagai contoh, tipe dari ekspresi $a + b$ ditentukan berdasarkan tipe dari a dan b . Jika keduanya bertipe Integer, hasilnya bertipe Integer; jika salah satunya bertipe Real, hasilnya dapat bertipe Real sesuai aturan kompatibilitas. Untuk operator relasional seperti $=$, $<>$, $<$, $<=$, $>$, dan $>=$, hasil ekspresinya adalah Boolean.

Tipe-tipe dasar (simple type) yang tersedia adalah sebagai berikut: Integer untuk bilangan bulat, Real untuk bilangan desimal, Char untuk satu karakter, Boolean untuk nilai kebenaran, dan String untuk teks. Selain tipe dasar, Arion juga mendukung Subrange, yaitu tipe yang membatasi himpunan nilai dari suatu tipe dasar tertentu, misalnya 1..10, serta Enumerated, yaitu tipe yang nilainya didefinisikan secara eksplisit melalui daftar identifier.

Secara konseptual, tipe dapat dipandang sebagai *type expression*. Type expression dapat berupa basic type, named type, atau tipe yang dibentuk menggunakan *type constructor*. Basic type mencakup Integer, Real, Boolean, Char, dan String. Named type adalah tipe yang diberi nama melalui deklarasi tipe. Sementara itu, type constructor digunakan untuk membentuk tipe yang lebih kompleks, seperti array, record, subrange, dan enumerated. Dengan ini, tipe-tipe *structured* dapat dianalisis sebagai struktur yang tersusun dari tipe-tipe lain.

Untuk tipe structured, berlaku aturan tambahan. Dua anonymous record dinyatakan *incompatible* meskipun memiliki struktur field yang identik, karena keduanya tidak berasal dari deklarasi tipe bernama yang sama. Dengan demikian, kompatibilitas record tidak hanya ditentukan oleh kesamaan struktur, tetapi juga oleh identitas tipe yang digunakan.

Tabel berikut merangkum aturan tipe yang digunakan untuk operator-operator dalam bahasa Arion.

Operator	Tipe Operand	Tipe Hasil
+, -, *	Integer atau Real	Integer atau Real (mengikuti operand)
/	Integer atau Real	Real
div, mod	Integer	Integer
=, <>, <, <=, >, >=	Integer, Real, Char, atau String	Boolean
not	Boolean	Boolean
and, or	Boolean	Boolean

Terdapat beberapa aturan khusus yang perlu diperhatikan. Operator pembagian (/) selalu menghasilkan tipe Real tanpa memandang tipe operandnya.

Operator *div* dan *mod* hanya dapat digunakan pada operand bertipe Integer. Ekspresi kondisi pada *if*-statement dan *while*-statement wajib bertipe Boolean. Tipe indeks pada array harus berupa simple type dan tidak boleh bertipe Real. Subrange juga tidak boleh memiliki tipe Real.

Type Compatibility Rule menentukan kapan dua tipe dianggap kompatibel dan kapan suatu tipe dapat diassign ke tipe lain. Dua tipe dinyatakan *compatible* jika: (1) keduanya merupakan tipe yang sama, (2) salah satu adalah subrange dari yang lain, atau keduanya merupakan subrange dari tipe yang sama, atau (3) keduanya bertipe String dengan panjang yang sama.

Sementara itu, suatu tipe T2 dinyatakan assignment-compatible dengan tipe T1 jika salah satu dari kondisi berikut terpenuhi: T1 adalah Real dan T2 adalah Integer; atau T1 dan T2 adalah *compatible*, bertipe Integer, Boolean, atau Char, dan nilai T2 merupakan subset dari nilai T1; atau T1 dan T2 adalah compatible dan bertipe String.

b. Symbol Table Management dan Declaration Checking

Symbol Table Management mencakup dua operasi utama: registrasi identifier saat deklarasi ditemukan, dan *lookup identifier* saat penggunaan ditemukan. Saat parser menemukan deklarasi variabel, konstanta, tipe, prosedur, atau fungsi, compiler memasukkan detail identifier tersebut ke dalam tabel tab dan memperbarui btab dari blok yang aktif.

Declaration checking memastikan dua aturan penting. Pertama, setiap identifier yang digunakan harus sudah dideklarasikan sebelumnya di scope yang dapat diakses. Pelanggaran aturan ini menghasilkan kesalahan *undeclared identifier*. Kedua, tidak boleh ada dua identifier dengan nama yang sama dideklarasikan dalam scope yang sama. Pelanggaran aturan ini menghasilkan kesalahan *redeclared identifier*. Dengan mekanisme ini, informasi yang diperoleh dari deklarasi tidak perlu disimpan langsung pada setiap token identifier di parse tree. Cukup dengan melakukan lookup, *semantic analyzer* dapat mengetahui apakah identifier sudah dideklarasikan, tipe apa yang dimilikinya, dan pada scope mana identifier tersebut berada.

c. Scope Resolution

Scope resolution adalah proses menentukan entri *symbol table* mana yang dimaksud oleh suatu penggunaan identifier. Proses ini penting karena identifier yang sama dapat dideklarasikan di beberapa scope yang berbeda dan yang berlaku adalah deklarasi di scope terdalam yang mengandung penggunaan tersebut (*shadowing*).

Algoritma lookup dimulai dari scope terdalam (lexical level saat ini) dan naik satu level setiap kali identifier tidak ditemukan, hingga mencapai scope global (level 0). Pada setiap level, penelusuran mengikuti rantai link pada tabel tab dari identifier terakhir yang dideklarasikan mundur ke identifier pertama dalam blok tersebut. Proses ini menjamin bahwa identifier dengan nama yang sama di scope yang lebih dalam akan menghalangi (*shadow*) identifier dengan nama yang sama di scope yang lebih luar.

Dalam bahasa dengan *lexical scoping*, sebuah identifier dapat dideklarasikan ulang pada scope yang lebih dalam selama tidak melanggar aturan *redeclaration* pada scope yang sama. Kondisi ini disebut shadowing. Shadowing terjadi ketika deklarasi identifier pada *inner scope* menutupi deklarasi dengan nama yang sama pada *outer scope*. Saat identifier tersebut digunakan, lookup akan menemukan deklarasi pada scope terdalam terlebih dahulu, sehingga deklarasi luar baru digunakan jika tidak ada deklarasi yang sesuai pada scope yang lebih dalam.

Sebagai contoh, jika variabel *x* dideklarasikan pada scope global dan kemudian identifier *x* dideklarasikan kembali di dalam suatu prosedur, maka penggunaan *x* di dalam prosedur akan merujuk pada deklarasi lokal. Sementara itu, penggunaan *x* di luar prosedur tetap merujuk pada deklarasi global. Oleh karena itu, *semantic analyzer* perlu membedakan antara *redeclaration* dalam scope yang sama, yang merupakan error, dan *shadowing* pada scope yang berbeda, yang dapat dianggap valid bergantung pada aturan bahasa.

d. Control Flow Validation

Control Flow Validation adalah proses pengecekan semantik terhadap struktur alur program. Berbeda dengan *syntax analysis* yang hanya memastikan

bentuk statement sesuai grammar, *control flow validation* memastikan bahwa komponen-komponen di dalam statement memiliki makna yang valid menurut aturan bahasa. Dengan demikian, statement seperti if-statement, while-statement, repeat-statement, dan for-statement tidak hanya harus tersusun secara sintaksis benar, tetapi juga harus memenuhi syarat semantik tertentu.

Pada Arion Compiler, validasi alur kontrol difokuskan pada konstruksi yang tersedia dalam grammar. Ekspresi kondisi pada if-statement, while-statement, dan repeat-statement harus menghasilkan nilai bertipe Boolean, karena ekspresi tersebut digunakan untuk menentukan percabangan atau pengulangan eksekusi. Jika ekspresi kondisi menghasilkan tipe lain, misalnya Integer, Real, Char, atau String, maka program dianggap memiliki kesalahan semantik meskipun secara sintaksis tetap valid.

Untuk for-statement, semantic analyzer perlu memastikan bahwa variabel iterator merupakan identifier yang telah dideklarasikan sebagai variabel, bukan konstanta, tipe, prosedur, atau fungsi. Selain itu, variabel iterator dan batas awal maupun batas akhir perulangan harus memiliki tipe yang kompatibel dengan Integer, misalnya Integer atau subrange dari Integer. Pengecekan ini diperlukan karena operasi perulangan for melakukan perubahan nilai iterator secara teratur dari nilai awal menuju nilai akhir.

Secara umum, compiler untuk bahasa pemrograman lain dapat melakukan validasi alur kontrol tambahan, seperti memastikan break dan continue hanya muncul di dalam loop, memastikan function memiliki return pada semua jalur eksekusi, atau memberikan peringatan untuk unreachable code. Namun, validasi tersebut hanya relevan apabila konstruksi bahasanya tersedia dalam grammar. Oleh karena itu, dalam cakupan Arion Compiler pada Milestone 3, *control flow validation* dibatasi pada pengecekan tipe kondisi dan validitas penggunaan variabel iterator pada struktur kendali yang didukung oleh grammar.

BAB 2

PERANCANGAN PROGRAM

2.1. Gambaran Umum Progam

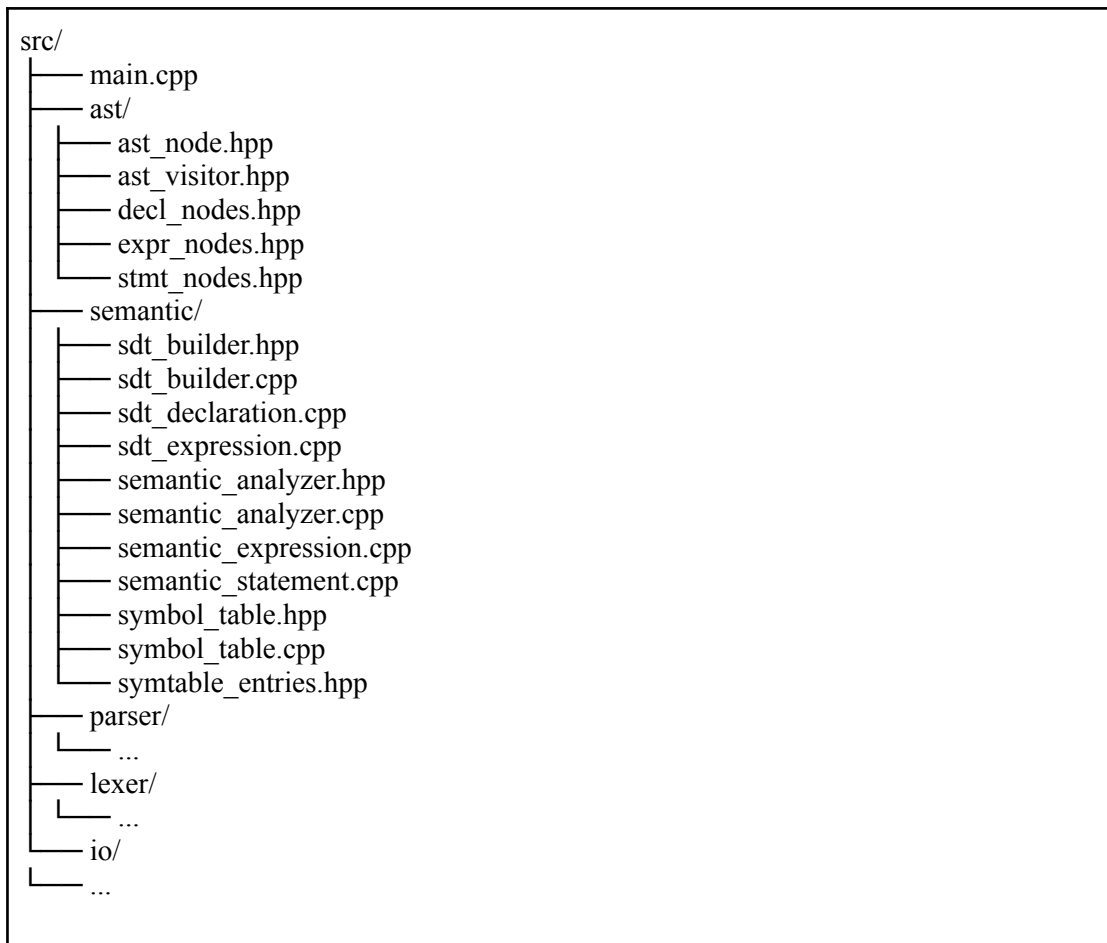
Pada Milestone 3 ini, kompilator diperluas dengan menambahkan modul Semantic Analysis. Program bertugas menerima masukan berupa parse tree yang dihasilkan oleh parser dari Milestone 2. Parse tree tersebut kemudian dikonversi menjadi Abstract Syntax Tree (AST) melalui Syntax-Directed Translation (SDT). Setelah itu, program akan menelusuri AST menggunakan metode Visitor Pattern untuk melakukan registrasi symbol table, pelacakan identifier (*lookup*), manajemen scope, dan evaluasi *Type Compatibility*. Hasil akhir dari program ini adalah Decorated AST yang telah terannotasi lengkap beserta struktur Symbol Table program tersebut.

2.2. Teknologi yang Digunakan

Sesuai dengan spesifikasi, pengembangan Semantic Analysis dilanjutkan menggunakan bahasa pemrograman C++ guna menjaga konsistensi dengan pengerjaan *lexer* dan *parser* pada milestone sebelumnya. Program memanfaatkan konsep *Object-Oriented Programming* (OOP) dengan menggunakan pola rancangan *Visitor Pattern* untuk menelusuri struktur data *Tree* secara rekursif, serta struktur *Stack* dan *Vector* standar dari C++ (STL) untuk manajemen pergerakan antar scope di dalam Symbol Table.

2.3. Struktur Program

Struktur direktori untuk milestone 3 adalah sebagai berikut,



2.4. Fungsi dan Kelas Utama

Kelas utama yang mengerjakan tahapan pada milestone 3 ini adalah SDTBuilder, SemanticAnalyzer, dan SymbolTable. Secara konseptual, SDTBuilder bertindak sebagai mesin translasi (*Syntax-Directed Translation*) yang mengubah struktur ParseNode hasil *parser* menjadi AstNode. Kelas ini secara rekursif membuang informasi sintaksis yang redundan (seperti tanda kurung atau *keyword* struktural) dan merakit *node* pohon baru yang murni berfokus pada logika operasional program. Setelah AST terbentuk, SemanticAnalyzer mengambil alih sebagai unit penelusur dengan arsitektur *Visitor Pattern*. Kelas ini menelusuri AST secara *depth-first search* (DFS) dari kiri ke kanan untuk mengevaluasi ruang lingkup (*scope*), memeriksa deklarasi ganda, serta memvalidasi kepatuhan operasi terhadap *Type Compatibility Rule*.

Untuk mendukung analisis tersebut, kelas SymbolTable bertindak sebagai pengelola memori dan hierarki deklarasi program. Struktur ini mengelola tiga tabel utama yang saling berelasi: tab (menyimpan informasi *identifier*, jenis objek, dan referensi tipe dasar), btab (menyimpan konteks blok *subprogram* serta ukuran variabel lokal), dan atab (mengelola batas indeks dan tipe elemen *array*). Fungsi pushBlock() dan popBlock() digunakan untuk mengatur tumpukan hierarki *scope* leksikal saat

penelusur masuk atau keluar dari sebuah blok prosedur. Pendaftaran variabel baru dilakukan melalui `enterTab()`, sedangkan pemanggilan fungsi `lookup()` akan memvalidasi *identifier* dengan cara menyusuri rantai deklarasi secara mundur (*bottom-up*) melalui atribut *link* dari *scope* saat ini hingga ke *scope* global.

Komponen penting berikutnya adalah struktur representasi pohon itu sendiri, yang terdiri dari kelas abstrak seperti `ExprNode`, `StmtNode`, dan `DeclNode` sebagai pewaris turunan `AstNode`. Berbeda dengan *parse tree* yang sekadar mewakili tata bahasa gramatikal, *node* pada AST merepresentasikan entitas logis yang independen (seperti `BinOpNode` untuk operasi matematika). Sebagai hasil akhir dari tahap analisis semantik, kelas-kelas ini telah membuang token *non-esensial* dan diperkaya dengan tambahan atribut krusial bernama `expression_type`. Atribut ini menyimpan indeks referensi tipe data dari *symbol table*, sehingga memastikan setiap *node* mengetahui secara pasti tipe hasil komputasinya.

Nama Fungsi	Keterangan
<code>build()</code>	Berada di <code>SDTBuilder</code> . Merepresentasikan pintu masuk translasi. Fungsi ini menerima <i>root</i> <code>ParseNode</code> dan merakitnya menjadi Abstract Syntax Tree menggunakan L-Attributed Grammar yang dievaluasi dari bawah ke atas.
<code>analyze()</code>	Berada di <code>SemanticAnalyzer</code> . Menjadi <i>public API</i> utama dari analisis semantik. Memulai proses <i>depth-first search traversal</i> ke dalam AST untuk melakukan deklarasi dan <i>type checking</i> .
<code>visit(ast::VarDeclNode)</code> , <code>visit(ast::ProcDeclNode)</code>	Fungsi khusus deklarasi. Bertugas mendaftarkan <i>identifier</i> beserta jenis objek dan tipe datanya ke dalam memori melalui <code>sym_table.enterTab()</code> . Jika memasuki prosedur atau fungsi, fungsi ini akan membuka <i>scope</i> baru dengan <code>pushBlock()</code> .
<code>visit(ast::AssignNode)</code> , <code>visit(ast::IfNode)</code> , <code>visit(ast::ForNode)</code>	Fungsi validasi <i>statement</i> . Pada <code>IfNode</code> dan <code>WhileNode</code> , diwajibkan <i>condition</i> bertipe boolean. Pada <code>AssignNode</code> dan <code>ForNode</code> , dievaluasi kesesuaian operannya dengan memanggil <i>helper</i> <code>isAssignmentCompatible()</code> .

visit(ast::BinOpNode), visit(ast::IdentNode)	Fungsi validasi dan inferensi ekspresi. visit(IdentNode) mencari validitas penggunaan variabel melalui lookup(). visit(BinOpNode) memastikan operasi matematika (+, -, *), relasional (=, >), dan logika (and, or) dikenakan pada operan yang sesuai, lalu menyimpan hasilnya pada expression_type.
---	---

2.5. Alur Kerja Program

Alur kerja *semantic analysis* dimulai setelah parser menghasilkan *parse tree*. Objek SDTBuilder menerima struktur tersebut dan mulai menerapkan *semantic action* pada setiap node. SDT mengeliminasi *keyword* terminal pembatas dan merakit simpul AST baru (seperti BinOpNode atau WhileNode) yang difokuskan pada operasi fungsional program. Setelah AST terbentuk penuh, objek SemanticAnalyzer dipanggil untuk menganalisis AST tersebut dari *root*. Sebelum melakukan perpindahan dari satu node ke node lainnya, sistem SymbolTable disiapkan dengan melakukan pendaftaran *predefined identifiers* seperti tipe data (Integer, Real, Boolean) serta beberapa *magic functions* bawaan bahasa.

Seiring penelusuran *SemanticAnalyzer* ke dalam deklarasi, program mendaftarkan nama konstanta, variabel, atau subprogram ke dalam *tab*. Saat memasuki *subprogram* atau iterasi dalam tubuh blok program utama, level *scope* dinaikkan sehingga tidak tercampur dengan variabel di luar blok tersebut. Dalam penelusuran operasi evaluasi (ekspresi matematika atau relasional), *analyzer* secara ketat memastikan tidak ada operasi antar tipe data yang menyalahi *Type Compatibility Rule* bahasa Arion. Jika terjadi pelanggaran, seperti variabel dipanggil tanpa deklarasi, array diindeks dengan *Real*, atau membedakan tipe string dengan int, *analyzer* menggunakan sistem *diagnostic* untuk mencetak pesan *error*. Setelah penelusuran selesai tanpa ada kesalahan, program sukses menghasilkan *Decorated AST* yang setiap nodenya telah diwarnai tipe data yang sesuai.

2.6. Aturan Semantic Action (Syntax-Directed Translation)

Dalam konversi *Parse Tree* menuju AST, kami memanfaatkan Aturan *Semantic Action* berikut:

- <assignment-statement> -> ID := <expr> : assignment_statement.node = new AssignNode (new VarNode(ID.lexeme), expr.node).
- <expression> -> <simple-expr> + <term> : expression.node = new BinOpNode('+', simple_expr.node, term.node).
- <factor> -> NUMBER : factor.node = new NumberNode(NUMBER.value).
- <factor> -> ID : factor.node = new VarNode(ID.lexeme).

- `<procedure-call> -> ID( <params> ) : procedure_call.node = new ProcCallNode(ID.lexeme, params.nodes).`

2.7. *Predefined Identifiers*

Sebelum traversal AST dimulai, struktur *Symbol Table* melakukan populasi entri awal untuk tipe bawaan. *Predefined Identifiers* yang kami definisikan adalah:

- Real, Integer, Char, Boolean, String (Sebagai identifier bertipe objek Type).
- True dan False (Memiliki tipe *Boolean* sebagai objek konstanta).
- `writeln` dan `readln` (Predefined identifier yang merepresentasikan sebuah prosedur Input/Output).

2.8. *Type Compatibility Rule*

Untuk memvalidasi interaksi antar-tipe selama perancangan program (seperti pada assignment dan operasi biner), kami menerapkan hukum *Type Compatibility* berikut:

- Dua buah *type* dikategorikan *compatible* (sesuai) apabila:
 - Mereka merupakan tipe primitif yang identik (Contoh: *Integer* dengan *Integer*).
 - Satu *type* adalah bentuk *subrange* dari tipe dasar (*parent*) lainnya, atau keduanya berasal dari dasar *subrange* yang sama.
 - Keduanya adalah tipe *String* yang mempunyai dimensi / limit karakter panjang yang ekuivalen.
- Evaluasi kesesuaian operasi *Assignment* ($X := Y$): Variabel T1 (*target*) dan T2 (*value*) bersifat assignment-compatible bila T1 bertipe *Real* dan T2 adalah *Integer*, ATAU tipe T1 identik kompatibel dengan tipe T2 dengan himpunan nilai T2 berada pada *subset range* T1 (jika menggunakan *Subrange*).

BAB 3

HASIL IMPLEMENTASI

3.1. Implementasi Struktur Data *Symbol Table*

Modul *Symbol Table* diimplementasikan menggunakan pendekatan array dinamis (`std::vector`) untuk menampung entri-entri identifier, array, dan blok. Struktur datanya didefinisikan secara rapi di dalam `syntable_entries.hpp`. Terdapat tiga struktur utama yang mewakili tabel yang saling berelasi: `TabEntry` (untuk identifier), `AtabEntry` (untuk tipe dan dimensi array), dan `BtabEntry` (untuk informasi lingkup subprogram atau record).

Di dalam `TabEntry`, atribut seperti `obj` (kelas objek), `type` (tipe dasar), `ref` (pointer ke tabel lain), dan `lev` (lexical level) disimpan dengan representasi integer atau *enum class* untuk memudahkan navigasi referensi.

```
60 struct TabEntry {
61     int idx;          // Name index, starts from 33 (after reserved keywords)
62     std::string id;   // The identifier name
63     int link;         // Pointer/index to the previous param/variable in the same scope
64     ObjClass obj;     // Object class (constant, var, function, etc.)
65     int type;         // Raw type of the identifier (bool, char, array, etc.)
66     int ref;          // Pointer/index to other table (atab/btab) if it's an
67     | | | | | | | // array/record/procedure block
68     int nrm;          // 1 normal, 0 parameter by ref
69     int lev;          // Lexical level: 0 global, increments for each block
70     int adr;          // Offset/address size in the stack frame
71 };
72
73 /**
74  * @brief Represents an array entry (atab)
75  *
76  */
77 struct AtabEntry {
78     int idx;          // Array entry index
79     int xtyp;         // Type of array index from tab
80     int etyp;         // Type of array element from tab
81     int eref;         // Pointer/index to other table if the element is composite
82     int low;          // Array lower bound index
83     int high;         // Array upper bound index
84     int elsz;         // Size of an element
85     int size;         // Total size of the array
86 };
87
88 /**
89  * @brief Represents a block/record entry (btab)
90  *
```

3.2. Implementasi Transformasi *Parse Tree* ke AST (SDT Builder)

Sebelum analisis semantik dijalankan, hasil dari parser (berupa *Parse Tree*) harus dikonversi menjadi *Abstract Syntax Tree* (AST) yang lebih ringkas dan berfokus

pada operasi komputasi. Kelas SDTBuilder pada file sdt_builder.cpp mengimplementasikan pola *Syntax-Directed Translation*.

Fungsi ini membaca ParseNode, membuang terminal yang tidak perlu, dan menyusun hierarki AstNode. Jika terjadi kesalahan struktur, modul ini berintegrasi langsung dengan Diagnoser untuk melaporkan *error* lokasi yang presisi.

```
void SDTBuilder::build() {
    built_ = true;
    has_errors_ = false;
    ast_root_.reset();

    if (parse_root_.type() != parser::NodeType::Program) {
        reportBuildError(parse_root_, "expected Program parse node as SDT root");
        return;
    }

    auto program = buildProgram(parse_root_);
    if (program == nullptr) {
        reportBuildError(parse_root_,
            "failed to translate parse tree into Program AST node");
        return;
    }

    ast_root_ = std::move(program);
}
```

3.3. Implementasi *Semantic Analyzer* Utama dan Manajemen *Scope*

Kelas utama yang memanipulasi *Symbol Table* dan mengevaluasi node AST adalah SemanticAnalyzer. Kelas ini diturunkan dari pola desain *Visitor*, yang menelusuri node-node AST secara *Depth-First*.

Manajemen *scope* (lingkup) dilekatkan pada penelusuran ini. Ketika *analyzer* masuk ke dalam sebuah deklarasi blok (BlockNode, ProcDeclNode, atau FuncDeclNode), metode enterScope() dipanggil untuk mendorong *lexical level* baru (menambah blok pada stack btab). Saat selesai, leaveScope() akan mengembalikannya.

```
SemanticAnalyzer::SemanticAnalyzer(const parser::ParseNode& parse_root,
                                   diag::Diagnoser& diagnoser)
    : sdt_builder_(parse_root, diagnoser), diagnoser_(diagnoser) {}

void SemanticAnalyzer::analyze() {
    sdt_builder_.build();
    if (sdt_builder_.hasErrors()) return;
    sdt_builder_.getAst().accept(*this);
}
```

```
SemanticAnalyzer::SemanticAnalyzer(const parser::ParseNode& parse_root,
                                   diag::Diagnoser& diagnoser)
    : sdt_builder_(parse_root, diagnoser), diagnoser_(diagnoser) {}

void SemanticAnalyzer::analyze() {
    sdt_builder_.build();
    if (sdt_builder_.hasErrors()) return;
    sdt_builder_.getAst().accept(*this);
}
```

3.4. Implementasi Evaluasi Deklarasi (Registrasi *Symbol Table*)

Pada bagian deklarasi program (seperti Variabel, Prosedur, dan Tipe), SemanticAnalyzer bertugas mendaftarkan setiap identifier yang sah ke dalam SymbolTable. Pada semantic_declaration.cpp, diimplementasikan pengecekan untuk menolak pendaftaran (*redeclaration error*) apabila identifier dengan nama yang sama sudah eksis di lingkup/scope yang sama.

Cuplikan kode berikut menunjukkan bagaimana variabel baru divalidasi dan di-insert ke dalam tab menggunakan fungsi lookupCurrentScope dan enterTab.

```

void SemanticAnalyzer::visit(ast::VarDeclNode& node) {
    auto* spec = dynamic_cast<ast::TypeSpecNode*>(node.type_spec.get());
    if (spec == nullptr) {
        reportError("invalid type specification in variable declaration");
        return;
    }

    const int var_type = resolveTypeSpec(*spec);
    node.expression_type = var_type;

    for (const auto& id : node.identifiers) {
        if (sym_table.lookupCurrentScope(id.lexeme)) {
            reportError("identifier '" + id.lexeme + "' already declared");
            continue;
        }

        sym_table.enterTab(id.lexeme, ObjClass::Variable, var_type);
    }
    // tab_index points to the first identifier registered for this declaration
    if (!node.identifiers.empty()) {
        if (auto entry = sym_table.lookup(node.identifiers[0].lexeme)) {
            node.tab_index = entry->idx;
        }
    }
}

```

3.5. Implementasi Pengecekan Tipe (*Type Checking*) pada Ekspresi

Evaluasi tipe komputasi dieksekusi secara *bottom-up* di dalam `semantic_expression.cpp`. Saat mengevaluasi operasi biner (`BinOpNode`), program pertama-tama akan mengevaluasi operan kiri dan operan kanan secara rekursif (melalui `accept(*this)`), lalu menganalisis interaksi tipe keduanya.

Kode di bawah menunjukkan implementasi ketat dari aturan *Type Compatibility*. Misalnya, operator aritmatika seperti `+`, `-`, dan `*` hanya mengizinkan bilangan numerik (Integer atau Real) dan akan mendikte bahwa jika salah satu operan adalah *Real*, maka tipe hasil ekspresi (sintesis) juga berujung menjadi *Real*. Sebaliknya, penggunaan operator AND/OR dipaksa khusus untuk tipe *Boolean*.

```

void SemanticAnalyzer::visit(ast::BinOpNode& node) {
    node.right->accept(*this);

    const int l = node.left->expression_type;
    const int r = node.right->expression_type;
    const int int_t = get_base_type("integer");
    const int real_t = get_base_type("real");
    const int bool_t = get_base_type("boolean");

    using T = lexer::TokenType;
    const T op = node.op.type;

    if (op == T::PLUS || op == T::MINUS || op == T::TIMES) {
        if ((l == int_t || l == real_t) && (r == int_t || r == real_t)) {
            node.expression_type = (l == real_t || r == real_t) ? real_t : int_t;
        } else {
            reportError("operands of '" + node.op.lexeme + "' must be numeric");
            node.expression_type = 0;
        }
    } else if (op == T::RDIV) {
        if ((l == int_t || l == real_t) && (r == int_t || r == real_t)) {
            node.expression_type = real_t;
        } else {
            reportError("operands of '/' must be numeric");
            node.expression_type = 0;
        }
    } else if (op == T::IDIV || op == T::IMOD) {
        if (l == int_t && r == int_t) {
            node.expression_type = int_t;
        } else {
            reportError("operands of '" + node.op.lexeme + "' must be integer");
            node.expression_type = 0;
        }
    }
}

```

BAB 4

PENGUJIAN

4.1. Pengujian ke-1

Input	Output
<pre> program VarAssign; var x, y: integer; r: real; begin x := 42; y := x + 1; r := 3.14; x := y * 2 - 1; end. </pre>	<pre> ast: ProgramNode('VarAssign') +-- VarDecl(x, y) -> type:33, tab:41 +-- VarDecl(r) -> type:34, tab:43 \-- Compound +-- Assign +-- Ident('x') -> type:33, tab:41 \-- Int(42) -> type:33 +-- Assign +-- Ident('y') -> type:33, tab:42 \-- BinOp('+') -> type:33 +-- Ident('x') -> type:33, tab:41 \-- Int(1) -> type:33 +-- Assign +-- Ident('r') -> type:34, tab:43 \-- Real(3.14) -> type:34 \-- Assign +-- Ident('x') -> type:33, tab:41 \-- BinOp('-') -> type:33 +-- BinOp('*') -> type:33 +-- Ident('y') -> type:33, tab:42 \-- Int(2) -> type:33 \-- Int(1) -> type:33 sym tab: tab: idx id obj type ref nrm lev adr link ----- ----- ----- ----- ----- ----- ----- ----- ----- 0 <reserved> keyword 0 0 0 0 0 0 1 program keyword 0 0 0 0 0 0 2 procedure keyword 0 0 0 0 0 0 3 function keyword 0 0 0 0 0 0 4 begin keyword 0 0 0 0 0 0 5 end keyword 0 0 0 0 0 0 6 if keyword 0 0 0 0 0 0 7 then keyword 0 0 0 0 0 0 8 else keyword 0 0 0 0 0 0 9 case keyword 0 0 0 0 0 0 10 while keyword 0 0 0 0 0 0 11 do keyword 0 0 0 0 0 0 12 repeat keyword 0 0 0 0 0 0 </pre>

	13 until keyword 0 0 0 0 0 0 14 for keyword 0 0 0 0 0 0 15 to keyword 0 0 0 0 0 0 16 downto keyword 0 0 0 0 0 0 17 var keyword 0 0 0 0 0 0 18 const keyword 0 0 0 0 0 0 19 type keyword 0 0 0 0 0 0 20 array keyword 0 0 0 0 0 0 21 record keyword 0 0 0 0 0 0 22 of keyword 0 0 0 0 0 0 23 boolean keyword 0 0 0 0 0 0 24 integer keyword 0 0 0 0 0 0 25 real keyword 0 0 0 0 0 0 26 char keyword 0 0 0 0 0 0 27 string keyword 0 0 0 0 0 0 28 not keyword 0 0 0 0 0 0 29 and keyword 0 0 0 0 0 0 30 or keyword 0 0 0 0 0 0 31 div keyword 0 0 0 0 0 0 32 mod keyword 0 0 0 0 0 0 33 integer type 33 0 1 0 1 0 34 real type 34 0 1 0 2 33 35 boolean type 35 0 1 0 3 34 36 char type 36 0 1 0 4 35 37 string type 37 0 1 0 5 36 38 true constant 35 0 1 0 1 37 39 false constant 35 0 1 0 0 38 40 VarAssign type 33 0 1 0 0 39 41 x variable 33 0 1 0 0 40 42 y variable 33 0 1 0 1 41 43 r variable 34 0 1 0 2 42 btab: idx last lpar psze vsze ---- ----- ----- ----- ----- 0 43 0 0 3 1 43 0 0 0 atab: idx xtyp etyp eref low high elsz size ---- ----- ----- ----- ----- ----- ----- -----
--	--

4.2. Pengujian ke-2

Input	Output
program Minimal; begin end.	ast: ProgramNode('Minimal') \-- Compound

sym tab:

tab:

idx	id	obj	type	ref	nrm	lev	adr	link
-----	-----	-----	-----	-----	-----	-----	-----	-----
0	<reserved>	keyword		0	0	0	0	0
1	program	keyword		0	0	0	0	0
2	procedure	keyword		0	0	0	0	0
3	function	keyword		0	0	0	0	0
4	begin	keyword		0	0	0	0	0
5	end	keyword		0	0	0	0	0
6	if	keyword		0	0	0	0	0
7	then	keyword		0	0	0	0	0
8	else	keyword		0	0	0	0	0
9	case	keyword		0	0	0	0	0
10	while	keyword		0	0	0	0	0
11	do	keyword		0	0	0	0	0
12	repeat	keyword		0	0	0	0	0
13	until	keyword		0	0	0	0	0
14	for	keyword		0	0	0	0	0
15	to	keyword		0	0	0	0	0
16	downto	keyword		0	0	0	0	0
17	var	keyword		0	0	0	0	0
18	const	keyword		0	0	0	0	0
19	type	keyword		0	0	0	0	0
20	array	keyword		0	0	0	0	0
21	record	keyword		0	0	0	0	0
22	of	keyword		0	0	0	0	0
23	boolean	keyword		0	0	0	0	0
24	integer	keyword		0	0	0	0	0
25	real	keyword		0	0	0	0	0
26	char	keyword		0	0	0	0	0
27	string	keyword		0	0	0	0	0
28	not	keyword		0	0	0	0	0
29	and	keyword		0	0	0	0	0
30	or	keyword		0	0	0	0	0
31	div	keyword		0	0	0	0	0
32	mod	keyword		0	0	0	0	0
33	integer	type		33	0	1	0	1
34	real	type		34	0	1	0	2
35	boolean	type		35	0	1	0	3
36	char	type		36	0	1	0	4
37	string	type		37	0	1	0	5
38	true	constant		35	0	1	0	1
39	false	constant		35	0	1	0	0
40	Minimal	type		33	0	1	0	0

btab:

| idx | last | lpar | psze | vsze |

	<pre> ---- ---- ---- ---- ---- 0 40 0 0 0 1 40 0 0 0 atab: idx xtyp etyp eref low high elsz size ---- ---- ---- ---- ---- ---- ---- ---- </pre>
--	--

4.3. Pengujian ke-3

Input	Output
<pre> program TypeRange; type Digit == 0..9; UpperAlpha == 'A'..'Z'; var d: Digit; begin d := 5; end. </pre>	<pre> .\test\milestone-3\semantic-test\10_type_range.in.txt: semantic error: subrange lower bound must be integer .\test\milestone-3\semantic-test\10_type_range.in.txt: semantic error: subrange upper bound must be integer .\test\milestone-3\semantic-test\10_type_range.in.txt: semantic error: assignment type mismatch </pre>

4.4. Pengujian ke-4

Input	Output
<pre> program ArrayDecl; var nums: array[1..5] of integer; grid: array[0..3] of real; i: integer; begin i := 1; nums[1] := 10; nums[i] := 20; grid[0] := 1.5; end. </pre>	<pre> ast: ProgramNode('ArrayDecl') +-- VarDecl(nums) -> type:42, tab:43 +-- VarDecl(grid) -> type:45, tab:46 +-- VarDecl(i) -> type:33, tab:47 \-- Compound +-- Assign +-- Ident('i') -> type:33, tab:47 \-- Int(1) -> type:33 +-- Assign +-- ArrayAccess -> type:33 +-- Ident('nums') -> type:42, tab:43 \-- Int(1) -> type:33 \-- Int(10) -> type:33 +-- Assign +-- ArrayAccess -> type:33 +-- Ident('nums') -> type:42, tab:43 </pre>

<pre> \-- Ident('i') -> type:33, tab:47 \-- Int(20) -> type:33 \-- Assign +-- ArrayAccess -> type:34 +-- Ident('grid') -> type:45, tab:46 \-- Int(0) -> type:33 \-- Real(1.5) -> type:34 </pre>									
sym tab:									
tab:									
idx	id	obj	type	ref	nrm	lev	adr	link	
0	<reserved>	keyword		0	0	0	0	0	0
1	program	keyword		0	0	0	0	0	0
2	procedure	keyword		0	0	0	0	0	0
3	function	keyword		0	0	0	0	0	0
4	begin	keyword		0	0	0	0	0	0
5	end	keyword		0	0	0	0	0	0
6	if	keyword		0	0	0	0	0	0
7	then	keyword		0	0	0	0	0	0
8	else	keyword		0	0	0	0	0	0
9	case	keyword		0	0	0	0	0	0
10	while	keyword		0	0	0	0	0	0
11	do	keyword		0	0	0	0	0	0
12	repeat	keyword		0	0	0	0	0	0
13	until	keyword		0	0	0	0	0	0
14	for	keyword		0	0	0	0	0	0
15	to	keyword		0	0	0	0	0	0
16	downto	keyword		0	0	0	0	0	0
17	var	keyword		0	0	0	0	0	0
18	const	keyword		0	0	0	0	0	0
19	type	keyword		0	0	0	0	0	0
20	array	keyword		0	0	0	0	0	0
21	record	keyword		0	0	0	0	0	0
22	of	keyword		0	0	0	0	0	0
23	boolean	keyword		0	0	0	0	0	0
24	integer	keyword		0	0	0	0	0	0
25	real	keyword		0	0	0	0	0	0
26	char	keyword		0	0	0	0	0	0
27	string	keyword		0	0	0	0	0	0
28	not	keyword		0	0	0	0	0	0
29	and	keyword		0	0	0	0	0	0
30	or	keyword		0	0	0	0	0	0
31	div	keyword		0	0	0	0	0	0
32	mod	keyword		0	0	0	0	0	0
33	integer	type		33	0	1	0	1	0
34	real	type		34	0	1	0	2	33
35	boolean	type		35	0	1	0	3	34
36	char	type		36	0	1	0	4	35

	<pre> 37 string type 37 0 1 0 5 36 38 true constant 35 0 1 0 1 37 39 false constant 35 0 1 0 0 38 40 ArrayDecl type 33 0 1 0 0 39 41 \$anonym_type0 type 6 0 1 0 0 40 42 \$anonym_type1 type 7 1 1 0 0 41 43 nums variable 42 0 1 0 0 42 44 \$anonym_type2 type 6 0 1 0 0 43 45 \$anonym_type3 type 7 2 1 0 0 44 46 grid variable 45 0 1 0 1 45 47 i variable 33 0 1 0 2 46 btab: idx last lpar psze vsze ----- ----- ----- ----- ----- 0 47 0 0 3 1 47 0 0 0 atab: idx xtyp etyp eref low high elsz size ----- ----- ----- ----- ----- ----- ----- ----- 1 41 33 0 1 5 1 5 2 44 34 0 0 3 1 4 </pre>
--	--

4.5. Pengujian ke-5

Input	Output
<pre> program FuncWithParams; function Add(a, b: integer): integer; begin Add := a + b; end; procedure PrintSum(x: integer; y: integer); begin writeln(Add(x, y)); end; begin PrintSum(3, 4); end. </pre>	<pre> ast: ProgramNode('FuncWithParams') +-- FuncDecl('Add') -> tab:41, params:1 \-- Compound \-- Assign +-- Ident('Add') -> type:33, tab:41 \-- BinOp('+') -> type:33 +-- Ident('a') -> type:33, tab:42 \-- Ident('b') -> type:33, tab:43 +-- ProcDecl('PrintSum') -> tab:44, params:2 \-- Compound \-- ProcCall('writeln') \-- FuncCall('Add') -> type:33, tab:41 \-- Compound \-- ProcCall('PrintSum') -> tab:44 +-- Int(3) -> type:33 \-- Int(4) -> type:33 </pre>

sym tab:								
tab:								
idx	id	obj	type	ref	nrm	lev	adr	link
----	-----	-----	-----	-----	-----	-----	-----	-----
-								
0	<reserved>	keyword		0	0	0	0	0
1	program	keyword		0	0	0	0	0
2	procedure	keyword		0	0	0	0	0
3	function	keyword		0	0	0	0	0
4	begin	keyword		0	0	0	0	0
5	end	keyword		0	0	0	0	0
6	if	keyword		0	0	0	0	0
7	then	keyword		0	0	0	0	0
8	else	keyword		0	0	0	0	0
9	case	keyword		0	0	0	0	0
10	while	keyword		0	0	0	0	0
11	do	keyword		0	0	0	0	0
12	repeat	keyword		0	0	0	0	0
13	until	keyword		0	0	0	0	0
14	for	keyword		0	0	0	0	0
15	to	keyword		0	0	0	0	0
16	downto	keyword		0	0	0	0	0
17	var	keyword		0	0	0	0	0
18	const	keyword		0	0	0	0	0
19	type	keyword		0	0	0	0	0
20	array	keyword		0	0	0	0	0
21	record	keyword		0	0	0	0	0
22	of	keyword		0	0	0	0	0
23	boolean	keyword		0	0	0	0	0
24	integer	keyword		0	0	0	0	0
25	real	keyword		0	0	0	0	0
26	char	keyword		0	0	0	0	0
27	string	keyword		0	0	0	0	0
28	not	keyword		0	0	0	0	0
29	and	keyword		0	0	0	0	0
30	or	keyword		0	0	0	0	0
31	div	keyword		0	0	0	0	0
32	mod	keyword		0	0	0	0	0
33	integer	type		33	0	1	0	1
34	real	type		34	0	1	0	2
35	boolean	type		35	0	1	0	3
36	char	type		36	0	1	0	4
37	string	type		37	0	1	0	5

	38 true constant 35 0 1 0 1 37
	39 false constant 35 0 1 0 0 38
	40 FuncWithParams type 33 0 1 0 0
	39
	41 Add function 33 0 1 0 0 40
	42 a variable 33 0 1 1 0 41
	43 b variable 33 0 1 1 1 42
	44 PrintSum procedure 0 0 1 0 0
	41
	45 x variable 33 0 1 1 0 44
	46 y variable 33 0 1 1 1 45
	btab:
	idx last lpar psze vsze
	---- ----- ----- ----- -----
	0 44 0 0 0
	1 43 0 0 2
	2 46 0 0 2
	3 44 0 0 0
	atab:
	idx xtyp etyp eref low high elsz size
	---- ----- ----- ----- ----- ----- ----- -----

Referensi

1. Spesifikasi Milestone 1 - Tubes IF2224 TBFO
2. Spesifikasi Milestone 2 - Tubes IF2224 TBFO
3. Spesifikasi Milestone 3 - Tubes IF2224 TBFO
4. <https://pdfs.semanticscholar.org/e142/91a9afec31014a9d489e3ad35a7a612088df.pdf>
5. https://www.tutorialspoint.com/compiler_design/pdf/compiler_design_syntax_analysis.pdf
6. <https://www.geeksforgeeks.org/compiler-design/recursive-descent-parser/>